

ReAct

ReAct (Reasoning + Action)

引子

传统RAG框架的架构是固定的：Query - Retrieval - Generation，这种普通的架构缺乏思考能力。

传统RAG的三大原罪就是如此：

- 1. 无法多跳推理
- 2. 不会判断信息是否充足
- 3. 模型只会脑补不会查资料

ReAct范式的出现，就是为了解决这个问题。

一：ReAct是什么？

1.1 一句话定义

ReAct = Reasoning + Acting (推理 + 行动)

目的是让大模型像人类一样：边想边做，做完再看效果，效果不好就调整。

1.2 生活案例：🎮

假设你正在玩一个 RPG 游戏，要打败一个 Boss：

- 1. **推理 (Thought)**：先判断 Boss 是什么属性？我现在装备够不够？
- 2. **行动 (Action)**：发现血量不够，先去商店买药水。
- 3. **观察 (Observation)**：买完药水后发现金币还剩很多，可以再升级武器。
- 4. **推理 (Thought)**：Boss 是火属性，我应该用冰属性武器更有效。
- 5. **行动 (Action)**：切换冰属性武器，开始攻击 Boss。
- 6. **观察 (Observation)**：Boss 掉血明显变快，但还有护盾。
- 7. **推理 (Thought)**：说明冰属性有效，下一步应该先破盾再输出。

所以，ReAct 的本质不是“直接给答案”，而是让 Agent 像人一样：

先判断情况 → 执行动作 → 看结果 → 根据结果调整策略

在 AI Agent 中：

- **Thought**：模型分析当前情况，决定下一步做什么
- **Action**：调用工具，比如搜索、查数据库、执行代码
- **Observation**：工具返回结果
- **再 Thought**：模型根据结果继续判断下一步

这就是 ReAct 为什么适合用来构建 Agent 的原因。

1.3 专业解释： TAO三步闭环机制

Thought-Action-Observation (TAO) 闭环机制实现动态交互：



相关论文：

《ReAct: Synergizing Reasoning and Acting in Language Models》

二：为什么非ReAct不可？

2.1 三种范式大乱斗

这张表主要对比了三种 Agent / LLM 工作范式：

维度	Chain-of-Thought (CoT)	Action-Only	ReAct（我们主角）
推理能力	✅ 内部推理链	❌ 直接行动	✅ 交替进行
行动能力	❌ 没有外部交互	✅ 调用工具	✅ 自主决策
与环境交互	❌ 闭门造车	✅ 但无推理	✅ 有反馈闭环
幻觉风险	🔴 高，容易自己瞎想	🟡 中等	🟢 低，基于实际结果

2.2 Chain-of-Thought (CoT)

CoT 的核心是：

让模型先一步步思考，再给出答案。

它的优点是有推理过程，适合数学题、逻辑题、分析题。

但是它的问题是：

只会在脑子里想，不能真正去外部世界查资料、调用工具或验证结果。

所以它容易出现“想得很完整，但事实不一定对”的情况，也就是幻觉风险较高。

2.3 Action-Only

Action-Only 的核心是：

不怎么思考，直接调用工具或执行动作。

它的优点是可以和外部环境交互，例如搜索、查数据库、调用 API。

但它的问题是：

缺少推理能力，不知道为什么要这样做，也不一定知道下一步该怎么调整。

它像是一个只会执行命令的工具人，能行动，但不够聪明。

2.4 ReAct

ReAct 的核心是：

Reasoning + Acting，也就是“推理”和“行动”结合。

它不是只思考，也不是只行动，而是：

先思考 → 再行动 → 观察结果 → 再继续思考 → 再行动

因此 ReAct 同时具备：

- **推理能力**：知道为什么要做某件事
 - **行动能力**：可以调用工具与外部环境交互
 - **反馈闭环**：能根据工具返回的结果调整下一步
 - **更低幻觉风险**：因为它不是纯靠想象，而是基于实际观察结果
-

如果把三种范式比作“做作业”：

CoT：自己在脑子里推理

你遇到一道题，会一步步分析：

这题应该先看条件，再套公式，再推结果。

但你不能查资料，也不能用计算器。

所以如果你记错了公式，就可能一本正经地算错。

Action-Only：直接查答案

你遇到题目后，马上去搜索或调用工具。

但你没有认真分析题目，也不知道哪个结果更可靠。

所以你虽然能拿到信息，但不一定会判断。

ReAct：边思考边查证

你先分析题目需要什么信息，然后去查资料或调用工具。

拿到结果后，再判断结果是否有用。

如果不够，再继续查、继续推理。

这就是最接近真实人类解决问题的方式。

2.4 总结

范式	核心特点	主要问题
CoT	会推理	不能和外部环境交互，容易幻觉
Action-Only	会行动	缺少推理，容易机械执行
ReAct	既会推理，又会行动	更适合构建 Agent

一句话总结：

CoT 是“只想不做”，Action-Only 是“只做不想”，ReAct 是“边想边做，并根据结果继续调整”。

2.5 Plan-and-Execute vs ReAct：谁是王者？

维度	Plan-and-Execute	ReAct
规划方式	先规划再执行，一次性全想好	边走边想，动态调整
适应性	较差，遇到意外就容易卡壳	极强，可以随时调整策略
适用场景	步骤固定的目标型任务	探索性、多步骤复杂任务
Token 效率	较高	较低，因为推理步骤更多

核心区别

Plan-and-Execute 更像是：

出发前把完整路线一次性规划好，然后严格按照计划执行。

它适合流程明确、步骤固定、不太容易发生变化的任务。

例如：

- 按模板生成报告
- 固定流程的数据处理
- 简单问答
- 步骤清晰的任务执行

但是它的问题是：

一旦中途遇到意外情况，原来的计划可能就不适用了。

ReAct 更像是：

一边执行，一边观察结果，再根据新情况调整下一步。

它适合不确定性更高、需要 动态判断 的任务。

例如：

- 多轮搜索
- 复杂问题分析

- 工具调用型 Agent
- 金融、法律、医疗等需要逐步校验的任务

ReAct 的优势在于它有反馈闭环，不是盲目执行，而是根据实际结果不断修正策略。

选择建议

任务类型	推荐范式
简单问答、步骤固定的任务	Plan-and-Execute
需要动态判断、可能遇到意外的任务	ReAct

一句话总结：

Plan-and-Execute 适合“路线已经很清楚”的任务，ReAct 适合“边走边探索”的任务。

2.4 企业价值：券商 AI Agent 的实践

图片中提到：

根据 2026 年行业报告，ReAct 架构成为金融 AI Agent 的标配。

在金融场景中，ReAct 的价值主要体现在：

- 可以把**合规校验**嵌入每一步执行环节
- 可以把**风险拦截**放入 Agent 的工具调用流程中
- 可以实现**全链路风险可追溯**
- 能够让 Agent 不只是“生成答案”，而是边执行边校验结果

券商场景案例

图片中举了两个例子：

1. **中泰证券、国元证券等券商的 AI Agent 系统**
采用 ReAct 架构，将合规校验和风险拦截嵌入每一步执行环节，实现全链路风险可追溯。
2. **Morgan Stanley 内部金融研究工作流的检索代理**
图片中提到其采用率高达 **98%**，准确率提升 **80%**。

为什么金融场景适合 ReAct？

金融业务不能只追求“回答快”，还需要：

- 每一步都有依据
- 每一步都能被检查
- 出错时能追溯原因
- 关键环节能进行风险控制
- 不能让模型纯靠想象输出结论

所以 ReAct 很适合**金融 AI Agent**，因为它不是一次性生成最终答案，而是：

先推理 → 调用工具 → 获取结果 → 校验结果 → 再继续推理

这使得整个执行过程更透明、更可控，也更适合企业级落地。

三：代码设计

3.1 tool 设计三板斧

```
# ❌ 错误示范：描述太模糊
@tool
def search(query):
    """搜索"""
    pass

# ✅ 正确示范：描述清晰、参数明确
@tool
def search_court_rulings(case_type: str, year: int, province: str) -> str:
    """检索法院判例

    根据案件类型、判决年份和省份检索相关判例。

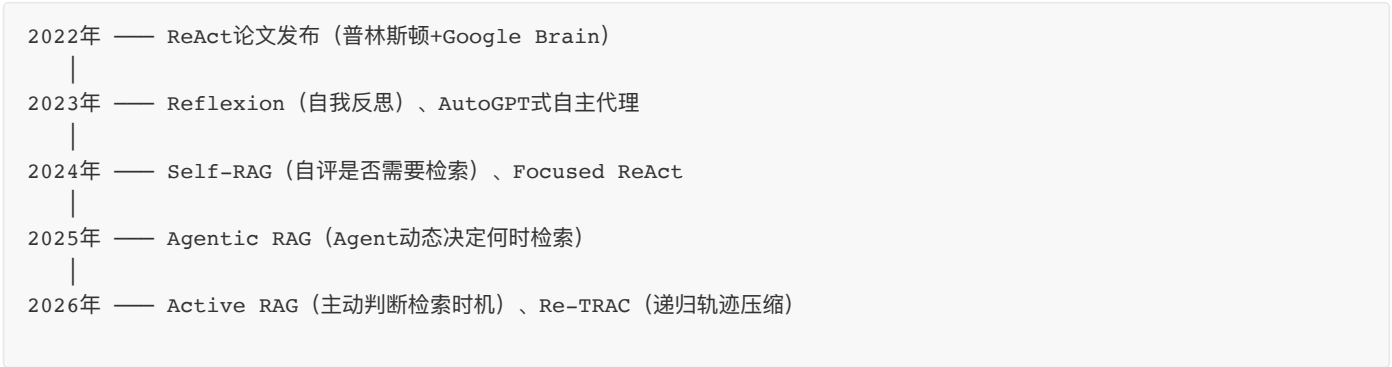
    Args:
        case_type: 案件类型，支持："民事"、"刑事"、"行政"、"劳动仲裁"
        year: 判决年份，格式YYYY，如2024
        province: 省份全称，如"广东省"、"浙江省"

    Returns:
        包含相关判例的列表，每条包含案号、判决要点和结果
    """
    pass
```

工具描述的黄金法则：

- 1. 动词开头：明确工具能做什么
- 2. 参数说明：每个参数的类型、格式、取值范围都要写
- 3. 返回格式：告诉LLM会得到什么格式的结果

2.2 ReAct的进化之路：从2022到2026 🔥



传统 RAG vs Agentic RAG：

能力	传统 RAG	Agentic RAG
检索策略	固定	Agent 动态决策
多跳推理	✗	✓
工具调用	无	API、SQL、图谱、外部搜索
失败处理	无感知	自动重试 / 换策略

Agentic RAG 工作流示例：

Query: "对比2023和2024年Q4销售数据，找出下滑超过20%的产品类别并分析原因"

Agent思考: 需要先拿两年的销售数据

↓

Action: SQL查询 2023 Q4数据

↓

Observation: [结果...]

↓

Action: SQL查询 2024 Q4数据

↓

Observation: [结果...]

↓

Action: 向量检索"产品类别下滑原因"相关文档

↓

Observation: [市场报告片段...]

↓

Agent综合分析 → 最终答案

Agentic RAG 更像是：

用户提问 → Agent 判断需要什么信息 → 选择工具 → 检索 / 查询 / 推理 → 如果失败就换策略 → 最后生成答案

💣 BUT! ! !

警告：约90%的Agentic RAG项目在生产部署中失败！

失败原因分析：

- 1. 链式失败的累积效应：每步95%成功率，10步后只有59.9%
- 2. 低估了错误处理的复杂度
- 3. 工具调用失败没有兜底方案

四：企业级实战指南（避坑版）

4.1 框架选型： LangChain vs LangGraph

维度	LangChain	LangGraph
上手难度	🌟 简单	🌟🌟🌟 较复杂
适用场景	快速原型	生产级应用
状态管理	基础	强大，图结构
并行执行	支持	原生支持
生产稳定性	🟡 一般	🟢 更好

选择建议

使用阶段	推荐框架	原因
学习阶段	LangChain	上手快，适合快速理解 LLM 应用、RAG、Tool Calling、Agent 基础流程
生产项目	LangGraph	更可控，适合复杂 Agent、多节点流程、状态管理和生产级部署

框架	核心特点	更适合
LangChain	简单、灵活、组件丰富	学习、Demo、快速原型
LangGraph	状态强、流程清晰、生产更稳	复杂 Agent、生产级应用

一句话总结：

学习 Agent 先用 LangChain 快速入门，真正做复杂生产级 Agent 时更推荐 LangGraph。

4.2 生产环境“五不要” ！

```
# ❌ 不要1：没有终止条件
while True:
    result = agent.step() # 危险！可能无限循环

# ✅ 要这样：设置最大迭代次数
max_steps = 10
for step in range(max_steps):
    result = agent.step()
    if result.is_finished:
        break

# ❌ 不要2：没有错误处理
def tool_that_fails():
    return api_call() # 可能超时/报错

# ✅ 要这样：完善的错误处理
@tool
def robust_api_call(query: str) -> str:
    """带重试的API调用"""
    for attempt in range(3):
        try:
            return api_call(query)
        except RateLimitError:
            time.sleep(60) # 等1分钟再试
```



```

        except Exception as e:
            return f"查询失败: {str(e)}"
        return "查询失败, 已达最大重试次数"

# ❌ 不要3: Token无限累积
history.append(result) # 无限增长!

# ✅ 要这样: 智能截断
MAX_HISTORY_TOKENS = 4000
if calculate_tokens(history) > MAX_HISTORY_TOKENS:
    history = summarize_and_compress(history)

# ❌ 不要4: 没有安全边界
@tool
def execute_sql(query: str) -> str:
    """直接执行SQL"""
    return db.execute(query) # 危险! SQL注入!

# ✅ 要这样: 参数化查询 + 权限控制
@tool
def query_sales_data(category: str, year: int) -> str:
    """查询销售数据 (只读权限) """
    # 使用预编译语句
    result = db.execute(
        "SELECT * FROM sales WHERE category = ? AND year = ?",
        (category, year)
    )
    return format_results(result)

```

4.3 安全合规 Checklist

```

"""
金融/医疗等高敏感场景的合规检查清单
"""

COMPLIANCE_CHECKLIST = {
    "🔒 数据隔离": [
        "高敏感数据在隔离环境中处理",
        "检索结果不包含PII信息"
    ],
    "📋 审计追溯": [
        "每次工具调用记录完整日志",
        "保留完整的Thought-Action-Observation链",
        "支持事后回溯和问题定位"
    ],
    "⚠️ 风险控制": [
        "高风险决策必须人工确认",
        "设置每日/每用户调用上限",
        "异常行为实时告警"
    ],
    "🏷️ 结果标识": [
        "AI生成内容有明显标识",
        "置信度低于阈值时提示用户核实"
    ]
}

```

4.4 性能优化实战

```
# 优化1: 并行检索
import asyncio

async def parallel_search(queries: list[str]) -> list[str]:
    """并行执行多个检索任务"""
    tasks = [search_tool(q) for q in queries]
    results = await asyncio.gather(*tasks)
    return results

# 优化2: 结果缓存
from functools import lru_cache

@lru_cache(maxsize=1000)
def cached_kb_search(query: str) -> str:
    """知识库检索结果缓存"""
    return knowledge_base.search(query)

# 优化3: 提前终止
def should_continue(state, max_steps=10) -> bool:
    """判断是否继续循环"""
    if state.step_count >= max_steps:
        return False
    if state.is_finished:
        return False
    if state.has_looped(): # 检测是否陷入循环
        return False
    return True
```
